

Aula 10 — Completando as regras do pedido

Como completar as operações de um pedido sem quebrar as responsabilidades que já construímos?

PONTO DE PARTIDA

O pedido já sabe incluir e fechar

```
pedido.adicionarItem(teclado, 2);  
pedido.adicionarItem(mouse, 1);  
pedido.fechar();
```

Depois de fechar, uma nova inclusão é recusada.

Mas ainda não conseguimos corrigir uma quantidade nem retirar uma linha.

NOVA NECESSIDADE

O cliente desistiu do mouse

Antes de fechar, o pedido contém:

Produto	Quantidade	Subtotal
Teclado	2	300.0
Mouse	1	80.0

O total é 380.0. Como retirar somente a linha do mouse?

ATIVIDADE

NOVA NECESSIDADE

“Remover um item” ainda deixa decisões

O item será identificado:

1. pelo produto?
2. pela referência de `ItemPedido` ?
3. pela posição na lista?

E remover significa diminuir uma unidade ou eliminar a linha inteira?

Uma linha por mesma referência de produto

Nesta versão:

- cada pedido possui no máximo uma linha para a mesma referência de Produto ;
- remover elimina a linha inteira;
- produto ausente preserva o pedido;
- pedido fechado não pode ser editado.

São escolhas deste modelo pequeno, não regras universais de pedidos.

ARMADILHA

ENCAPSULAMENTO

A lista não é a API do pedido

```
pedido.getItems().remove(...);
```

Se o cliente receber a lista interna, poderá mudar o pedido sem passar pelas regras de fechamento ou de uma linha por produto.

O cliente solicita uma operação de domínio; `Pedido` decide se sua coleção muda.

IDENTIFICAÇÃO

Mesmo estado não significa mesma identidade

```
Produto mouse = new Produto("Mouse", 80.0);  
pedido.adicionarItem(mouse, 1);  
  
Produto outroMouse = new Produto("Mouse", 80.0);  
pedido.removerItem(outroMouse);
```

As duas chamadas a `new Produto(...)` criam o mesmo produto para este pedido?

ATIVIDADE
IDENTIFICAÇÃO

Preveja a remoção

A chamada `pedido.removeItem(outroMouse)` deve remover a linha do mouse?
Justifique usando identidade e referências.

IDENTIFICAÇÃO

A linha é localizada pela mesma referência

Não. `mouse` e `outroMouse` têm identidades diferentes.

```
pedido.removeItem(mouse);
```

Nesta versão, é a mesma referência usada na inclusão que identifica a linha.

O item responde à pergunta necessária

```
public boolean representa(Produto produto) {  
    return this.produto == produto;  
}
```

`ItemPedido` não entrega seu campo `produto`.

Ele responde somente se representa a referência que `Pedido` está procurando.

Pedido percorre a coleção que mantém

```
public void removerItem(Produto produto) {  
    if (!fechado) {  
        for (int indice = 0; indice < itens.size(); indice++) {  
            ItemPedido item = itens.get(indice);  
  
            if (item.representa(produto)) {  
                itens.remove(indice);  
                return;  
            }  
        }  
    }  
}
```

Java em foco — remover durante o percurso

O índice existe apenas dentro de `Pedido` :

- `itens.size()` informa quantas posições há;
- `itens.get(indice)` obtém a referência daquela posição;
- `itens.remove(indice)` remove a posição encontrada;
- `return` encerra a operação após a remoção.

Índice navega na lista; não identifica uma linha para o cliente.

Conceito-chave — operação do conjunto

Quem mantém uma coleção:

- localiza a parte sobre a qual a operação atua;
- decide se a mudança estrutural é permitida;
- preserva as regras do conjunto.

A parte localizada continua responsável por seu próprio estado.

ALTERAÇÃO

Agora precisamos corrigir uma quantidade

O pedido encontra a linha do teclado.

Quem deve guardar a nova quantidade?

```
// proposta inadequada, dentro de Pedido  
item.quantidade = novaQuantidade;
```

Escolha a responsabilidade

Compare:

```
item.quantidade = novaQuantidade;
```

```
item.alterarQuantidade(novaQuantidade);
```

Qual preserva o encapsulamento? Por quê?

ALTERAÇÃO

A quantidade continua pertencendo ao item

```
public void alterarQuantidade(int novaQuantidade) {  
    if (novaQuantidade > 0) {  
        quantidade = novaQuantidade;  
    }  
}
```

ItemPedido protege a quantidade positiva e continua calculando seu subtotal.

Zero muda a estrutura do pedido

Para a Versão 10:

Nova quantidade	Consequência
positiva	<code>ItemPedido</code> atualiza sua quantidade
zero	<code>Pedido</code> remove a linha
negativa	o estado é preservado

A regra de zero pertence a `Pedido`, porque remove uma parte de sua coleção.

Pedido coordena sem escrever no item

```
if (item.representa(produto)) {  
    if (novaQuantidade == 0) {  
        itens.remove(indice);  
    } else {  
        item.alterarQuantidade(novaQuantidade);  
    }  
    return;  
}
```

Pedido decide o efeito estrutural. ItemPedido decide se aceita a quantidade.

Preveja três consequências

Pedido aberto: teclado $\times 2$ a 150.0; mouse $\times 1$ a 80.0.

1. alterar teclado para 3;
2. alterar mouse para 0;
3. alterar teclado para -1.

Quais totais resultam em cada cenário independente?

ALTERAÇÃO

Os resultados mostram regras diferentes

1. teclado para 3 → **530.0**;
2. mouse para 0 → **300.0**;
3. teclado para -1 → **380.0**.

Quantidade zero remove uma linha. Quantidade negativa não altera o estado.

FECHAMENTO

Fechar deve proteger qual mudança?

A Aula 09 já decidiu:

| pedido fechado não recebe novas inclusões.

Agora remover e alterar quantidade também mudam estrutura ou total.

Devem continuar permitidas depois de `fechar()` ?

Localize a guarda correta

Qual proposta protege a regra para qualquer cliente?

1. `Main` consulta o fechamento antes de cada chamada;
2. `Pedido` consulta `fechado` em suas operações de edição;
3. `ItemPedido` recebe um `Pedido` para consultar o fechamento.

O pedido fechado preserva seu estado

A proposta 2 protege toda solicitação de edição.

- `Main` é só um cliente e pode esquecer a regra;
- `ItemPedido` não precisa conhecer o pedido;
- `Pedido` já conhece o fechamento e a coleção.

Depois de fechado, não adiciona, remove nem altera quantidades.

Preveja o resultado depois de fechar

```
pedido.adicionarItem(teclado, 2);  
pedido.adicionarItem(mouse, 1);  
pedido.fechar();  
  
pedido.removerItem(mouse);  
pedido.alterarQuantidade(teclado, 3);  
System.out.println(pedido.calcularTotal());
```

Qual valor é exibido? Quantas linhas permanecem?

FECHAMENTO

Nenhuma edição posterior é aceita

380.0

As duas linhas permanecem: teclado ×2 e mouse ×1.

Fechar pertence a cada objeto `Pedido` ; outro pedido aberto continua editável.

O que realmente mudou?

Parte	Responsabilidade final
Produto	descrição e preço
ItemPedido	produto, quantidade válida e subtotal
Pedido	coleção, localização e regras de edição
cliente	solicita operações e observa efeitos

`calcularTotal()` continua delegando subtotais aos itens.

Para levar ao Laboratório 10

- localizar a linha pela mesma referência de `Produto` ;
- manter a coleção privada em `Pedido` ;
- delegar a quantidade válida para `ItemPedido` ;
- interpretar quantidade zero como remoção;
- preservar itens e quantidades depois de fechar.

Completar o modelo é acrescentar operações sem deslocar as responsabilidades que já funcionam.